

Event Kameraları ile Küçük İHA Tespiti

Yiğit Kaya Bağcı

Özet

Bu rapor, küçük İHA tespiti üzerine yürütülen çalışmaları özetlemekte; özellikle event kameraların bu problem üzerindeki olumlu etkisine, EV-UAV [1] veri setine, EV-SpSegNet mimarisine ve küçük drone'lara ve ev-drone-detector reposuna ve sonuçların incelenip, tracking mimarisine entegresine odaklanmaktadır.

1 Giriş

Küçük İHA'ların hızla yaygınlaşması, karşı-İHA algılamasını güvenlik açısından kritik bir problem hâline getirdi. Sıradışı İHA hedefleri klasik RGB kameralar için zordur: çok küçük görünürler, hızlı hareket ederler, parlak bir gökyüzü önünde silikleşirler ve sıradan pozlama süreleriyle motion blur'a maruz kalırlar [2]. Olay kameraları, her piksel için log-skala parlaklık değişimini mikrosaniye çözünürlüğü ve 120 dB+ dynamic range ile asenkron olarak raporlayan bio-esinlenmeli sensörlerdir [2]; statik arka planı bastırır, motion blur'u neredeyse tamamen ortadan kaldırır ve hızla hareket eden bir İHA'yı uzay-zaman hacminde keskin ve seyrek bir yörünge olarak ortaya çıkarırlar. Standart CMOS'un sınırlı dinamik aralığının aksine parlak gökyüzü önünde saturasyona düşmez, gölge altında da yeterli kontrast üretirler. Hareketsiz sahne kısımları veri üretmediği için güç tüketimi düşük kalır; bu seyreklik aynı zamanda doğal bir attention mekanizması olarak hesaplama kaynaklarını hedefe yoğunlaştırır ve gerçek-zamanlı, düşük gecikmeli karşı-İHA sistemleri için ideal bir özellik kümesi sunar [2].

Son dönem yaklaşımlar içinde EV-SpSegNet [1] öne çıkmaktadır. Bu yöntem, event'leri 2B frame'lere yığmak yerine event akışını 3B point cloud olarak ele alır ve hedefin çizdiği sürekli yörüngeyi *segmente* eder; böylece veri sparsity'si korunur ve frame-tabanlı detector'ların kaybettiği ince temporal ipuçları kullanılabilir kalır.

Yayınlanan EV-SpSegNet pipeline'ının dezavantajı, girdisini **8 saniyelik** event window'lar halinde işlemesidir; bu, offline değerlendirme için kabul edilebilir, ancak karşı-İHA sistemlerinin düşük gecikme gereksinimleriyle çelişir. Rapor; yöntemin *ev-drone-detector* reposundaki uygulamasını, training ve inference'ın uçtan uca çalışması için yapılan küçük düzeltmeleri ve FRED tracker ile gerçek-zamanlı bir sistemde entegrasyon stratejilerini ele almaktadır.

Görsel yer tutucusu: Aynı İHA için event point cloud ile RGB frame'in yan yana örneği.

Şekil 1: RGB kameralar nesneleri yalnızca normal ışık altında görebilirken, event kameraları çok zorlu ışık koşullarında bile çalışabiliyor. Ayrıca event kamerası, küçük nesnelerin kesintisiz hareket rotasını kırmızı bir eğri şeklinde yakalayabiliyor.

2 Arka Plan ve İlgili Çalışmalar

2.1 Event kameraları ve küçük hedef tespiti

Olay tabanlı İHA tespiti üzerine yapılan kapsamlı bir değerlendirme [2], mevcut çalışmaları event temsiline göre dört ana aileye ayırmaktadır: (i) sabit bir Δt üzerinde event’leri biriktirip elde edilen *event frame*’leri üzerinde hazır 2B detector’ları (YOLO, DETR, Faster R-CNN) çalıştıranlar; (ii) zaman eksenini kaba bir biçimde koruyan voxel grid’leri (RVT vb.); (iii) time surface gibi *zaman-koruyan frame* temsilleri; ve (iv) asenkron akışın kendisi üzerinde çalışan saf point-cloud yöntemleri (KPConv, RandLA-Net, EV-SpSegNet vb.).

2.2 EV-UAV benchmark’ı

EV-UAV [1], bilğimiz dahilinde küçük İHA tespiti için özel olarak tasarlanan ilk yalnızca-event veri kümesidir. Kayıtlar bir tripod üzerine monte edilmiş DAVIS346 kamerası (346×260) ile ve bir yıldan uzun bir süre boyunca alınmıştır. Veri kümesi 99 train, 24 validation ve 24 test olmak üzere 147 sequence, 2,3 milyondan fazla event-level annotation ve ortalama $6,8 \times 5,4$ piksel boyutunda hedef içerir; bu, önceki olay tabanlı İHA veri kümelerinin yaklaşık 1/50’sidir. Özellikle dikkat edilmesi gereken nokta, etiketlerin paralel bir RGB kameradan taşınan bbox annotation’ları değil, doğrudan *event-level* oluşudur.

2.3 EV-SpSegNet: frame yerine yörünge segmentasyonu

EV-SpSegNet’in [1] temel gözlemi, (x, y, t) hacminde küçük hareketli bir İHA’nın *uzun ve sürekli bir yörünge* oluşturması; arka planın geniş yüzeyler, gürültünün ise yalıtılmış noktalar oluşturmasıdır. EV-SpSegNet bu geometrik prior’u üç bileşenle değerlendirir:

- **Voxelization.** Event’ler $E = \{(x_i, y_i, t_i, p_i)\}$ mean-pooling ile $1 \text{ px} \times 1 \text{ px} \times 1 \text{ ms}$ voxel’lere dönüştürülür. 8 sn’lik bir window’da bu, $352 \times 288 \times 8192$ şeklinde padded bir uzay verir.
- **GDSCA modülü ile sparse U-Net.** spconv sparse 3D convolution’ları üzerinde kurulan simetrik bir encoder/decoder; her aşamada bir Grouped Dilated Sparse Convolution Attention (GDSCA) modülü kullanır. GDSCA girdiyi kanallar boyunca böler, $\{1, 2, 3, 4\}$ dilation oranlarıyla sparse convolution’lar uygulayarak çok-ölçekli yerel yapıyı yakalar, sparse bir squeeze-and-excitation bloğuyla bunları birleştirir ve son olarak bir *patch-attention* katmanı ile global context’i dahil eder.
- **STC loss.** Pixel-level olmayan, ağı yoğun spatiotemporal komşuluk içeren event’leri korumaya, yalıtılmış noise event’lerini ise atmaya teşvik eden bir loss fonksiyonu.

EV-UAV üzerinde rapor edilen sonuçlar [1], EV-SpSegNet’i ele alınan 13 çalışmanın hepsinin önüne yerleştirir; aralarında frame-tabanlı detector’lar (SSD, Faster-RCNN, Deformable-DETR, YOLOv10), recurrent voxel-grid detector’ları (RVT, RED, SAST), spiking neural network detector’ları (EMS-YOLO, Spike-YOLO) ve point cloud baseline’ları (KPConv, RandLA-Net, COSeg) bulunur. Daha da önemlisi, EV-SpSegNet *en hızlısıdır*: bir RTX 3090 üzerinde 8 sn’lik event verisini yalnızca 4,0 M parametre ile 35,9 ms’de işler.

3 ev-drone-detector Uygulaması

ev-drone-detector reposu, EV-SpSegNet’in açık kaynaklı bir yeniden uygulamasıdır ve detection-tracking tabanlı bir pipeline’ın “ilk detector” aşaması olarak kullanılmak üzere hafifçe yeniden düzenlenmiştir. Klasör yapısı şöyledir:

Yöntem	Temsil	IoU $\uparrow$	ACC $\uparrow$	$P_d\uparrow$	$F_a(10^{-4})\downarrow$
YOLOv10-S	Event count	32.55	33.39	32.18	589.67
RVT	Voxel grid	43.21	51.38	60.35	55.68
KPConv	Point	48.19	57.28	68.59	16.32
RandLA-Net	Point	50.32	59.29	70.56	6.95
EV-SpSegNet	Point	55.18	65.02	77.53	1.63

Tablo 1: EV-UAV benchmark’ından seçilmiş sonuçlar (tam tablo [1]’da). EV-SpSegNet tüm metriklerde önde, false alarm rate’inde oldukça açık önde.

```

1 ev-drone-detector/
2 |-- configs/default.yaml          # Tüm hiper-parametreler
3 |-- src/ev_drone_detector/
4 |   |-- data/
5 |   |   |-- event_repr.py        # Voxelization, batch collation
6 |   |   '-- dataset.py           # EV-UAV loader + sentetik veri
7 |   |-- models/
8 |   |   |-- spgnet.py             # GDSCA’lı sparse U-Net
9 |   |   |-- blocks.py            # GDBlock, Sp-SE, temel bloklar
10 |   |   '-- patch_attention.py
11 |   |-- losses/stc_loss.py        # Spatiotemporal Correlation loss
12 |   |-- detection/
13 |   |   |-- detector.py          # End-to-end inference
14 |   |   '-- clustering.py        # DBSCAN -> bbox’lar
15 |   '-- utils/                   # Konfig, eval, viz
16 |-- frame_detector/              # FRED’e koprusu (frame detector)
17 |-- scripts/{train,detect}.py
18 '-- tests/                       # Birim testler

```

3.1 Voxelization

Saf-PyTorch voxelizer (`src/ev_drone_detector/data/event_repr.py`), orijinal projedeki özel CUDA kernel’ini değiştirir. Her event tek bir (b, x, y, t) anahtarına eşlenir, aynı anahtarlar mean-pooling’e sokulur ve sonuç `spconv.SparseConvTensor`’a sarılır. Tensor ile birlikte dönen `p2v_map`, her giriş event’i için son voxel indisini tutar; hem loss hem de downstream clustering için gereklidir.

3.2 SPGNet modeli

`src/ev_drone_detector/models/spgnet.py` U şeklindeki sparse 3D ağı uygular. Encoder, zaman eksenini üzerinde $[2, 2, 4]$ stride’lı dört `SparseConv3d` aşaması çalıştırır; her aşamanın ardından GDSCA gelir. Decoder, `SparseInverseConv3d` ile encoder’ı yansıtır ve channel reduction’ı, orijinal codebase ile eşleşmeyi korumak için, öğrenilen 1×1 convolution’lar yerine *reshape + sum* ile yapar. Output head, voxel başına “hedef olasılığı” üreten tek bir `Linear` $\rightarrow \sigma$ ’dır.

Birlikte gönderdiğimiz konfigürasyonda taban kanal genişliği $w = 12$, input kanalları (x_n, y_n, t_n, p) için 4, dilation oranları $\{1, 2, 3, 4\}$, spatial shape ise DAVIS346 sensörünün stride kademesi için padded $[352, 288, 8192]$ olarak ayarlanmıştır.

3.3 Spatiotemporal Correlation (STC) Loss

`src/ev_drone_detector/losses/stc_loss.py` loss’u, ana codebase’in sadık bir aktarımıdır. Sezgi şudur: $k \times k \times \tau$ neighborhood’unda yüksek güvenli komşusu çok olan bir event, aynı gerçek İHA yörüngesi üzerinde olma ihtimali açısından, aynı piksel tahminine sahip yalıtılmış bir noise event’inden çok daha yüksektir. Pixel-level bir BCE bu iki durumu ayırt edemez; STC ayırt eder.

Açıkçası, voxel başına tahminler p ve binary etiketler y verildiğinde, sabit bir spatiotemporal yoğunluk hesaplanır:

$$\rho(v) = \sum_{e_j \in V_{k\tau}(v)} p_j, \quad V_{k\tau}(v) = \text{voxel } v \text{ etrafındaki } k \times k \times \tau \text{ window.} \quad (1)$$

Bu, tüm ağırlıkları bir olan, sabit (öğrenilemez) bir sparse 3D convolution olarak uygulanır. Yoğunluk merkezlenir ve squash edilir:

$$w_{\text{stc}}(v) = \sigma(\rho(v) - \bar{\rho}), \quad (2)$$

ve aşağıdaki binary cross-entropy üzerinde event başına bir çarpan olarak kullanılır:

$$\mathcal{L}_{\text{stc}}(p, y) = \begin{cases} -w_{\text{stc}} \log p, & y = 1, \\ -(1 - w_{\text{stc}}) \log(1 - p), & y = 0. \end{cases} \quad (3)$$

w_{stc} `detach` edilerek autograd graph'ından koparılır; gradient yalnızca prediction terimine akar. Ana ayarlarla uyumlu olarak $k = 3$ ve $\tau = 5$ kullanırız.

Yan gözlem olarak, ablation çalışmalarının [1] gösterdiği gibi, $\mathcal{L}_{\text{stc}}$ mimariden bağımsız genelleşir: KPConv, RandLA-Net ve COSeg'e takıldığında hem IoU hem de false alarm rate'inde tutarlı bir iyileşme sağlar. Bu, STC'nin kodladığı *geometrik prior*'un, yani “küçük hedefler sürekli yörüngelerdir, gürültü değil” fikrinin, GDSCA backbone'undan daha taşınabilir bir katkı olduğunu gösterir.

3.4 Segmentation'dan Bounding Box'a Dönüşüm

EV-SpSegNet event-level segmentation üretirken, downstream tracker'lar bounding box girdisi bekler. Bu dönüşüm `detection/clustering.py` modülünde şu adımlarla gerçekleştirilir: voxel başına sigmoid skorları $\tau_{\text{seg}} = 0,5$ ile threshold'lanır; hayatta kalan event'ler (x, y) image plane'ine izdüşürülür; DBSCAN ($\varepsilon = 10$ px, $\text{min}_{\text{samples}} = 3$) ile kümelenir; en az 5 noktalı her cluster için 5 px padding'li axis-aligned bbox üretilip sensör çözünürlüğüne clip'lenir. Çıktı, ortalama confidence'a göre sıralanmış en fazla 10 detection'dır.

3.5 FRED için Frame Detector Köprüsü

`frame_detector/` modülü, EV-UAV `.npz` akışlarını ~ 30 FPS'te 2B event frame'lerine dönüştürerek FRED üzerinde eğitilmiş frame detector'ların EV-UAV üzerinde re-training'siz değerlendirilmesine olanak tanır [2]. İki kanal formatı desteklenmektedir: doğrudan görselleştirilebilen `uint8 H \times W \times 3 polarity_rgb` ve iki kanallı event-native float tensor `counts_2ch`. Bu modül, mevcut FRED pipeline'ı ile doğal entegrasyon noktasını oluşturmaktadır.

4 Mühendislik Bug Fix'leri

`ev-drone-detector`'ın end-to-end çalışması için yapılan başlıca düzeltmeler aşağıda listelenmiştir:

- `detector.py` ve `train.py`: `spconv 2.x`'te bulunmayan `voxel_tensor.to(device)` çağrısı yerine `event_repr.py`'deki `sparse_to_device(...)` helper'ı kullanıldı; aynı düzeltme training tarafında `collate_events` çıktısı için de uygulandı.
- `frame_detector/make_motion_video.py`: `.npz` sequence'lerinden motion video üreten bir sanity-check aracı eklendi.

Bu fix'lerin ardından tek bir NVIDIA GPU üzerinde training beklendiği gibi yakınsar; varsayılan `configs/default.yaml` ile (50 epoch, batch 1, Adam, $\eta = 10^{-3}$ scheduled decay) tam bir training run'ı, referans çalışmada [1] raporlanan IoU / P_d değerlerini benchmark hatası payı içinde yeniden üretir.

5 Gerçek-Zamanlı Çalıştırma Düşünceleri

Yayınlanmış hâliyle EV-SpSegNet bir **batch** yöntemdir: 8 saniyelik event buffer üzerinde tek bir segmentation üretir. Ağ RTX 3090’da 35,9 ms’de çalışsa da 8 saniyelik window, sistem doğrudan kullanıldığında sabit 8 saniyelik bir *end-to-end latency tabanı* yaratır. Bir İHA 8 saniye içinde kayda değer mesafe alır. Aşağıda üç gerçek-zamanlı çalıştırma stratejisi tartışılmaktadır.

5.1 Strateji A: Cold-start

EV-SpSegNet sequence başında bir kez çalıştırılır; çıkan bounding box hızlı bir 2B tracker’a devredilir (**frame_detector**/ köprüsü FRED-eğitilmiş tracker ile doğrudan uyumludur). Sonraki window’larda sistem yalnızca tracker üzerinden 30 FPS çalışır.

Artılar. 8 saniyelik bütçe yalnızca bir kez ödenir; steady-state throughput maksimumdur ve EV-SpSegNet en güçlü olduğu *discovery* görevine odaklanır.

Eksiler. Sistem ilk 8 saniye boyunca kördür; hedef kaybedildiğinde recovery mekanizması olmadığı için aynı bekleme tekrarlanır.

5.2 Strateji B: Track kaybında re-acquisition

2B tracker sürekli çalıştırılırken paralel olarak 8 saniyelik bir *ring buffer*’da ham event akışı tutulur. Tracker confidence’ı belirlenmiş bir threshold’un altına düştüğünde EV-SpSegNet buffer üzerinde çalıştırılıp hedef yeniden yakalanır ve tracker tekrar seed’lenir. Yapı, ByteTrack tarzı tracking-by-detection pipeline’larına benzer; ancak re-finder rolünde event-level bir segmentation modeli kullanılır.

Kritik nokta şudur: ring buffer her an $[t - 8\text{ sn}, t]$ aralığını barındırır, dolayısıyla re-acquisition için yeni bir 8 saniye *beklenmez*. Örneğin tracker $t = 72\text{ sn}$ ’de hedefi kaybederse, EV-SpSegNet aynı anda buffer’da hazır bulunan $[64, 72]\text{ sn}$ penceresinde çalıştırılır ve sistem yalnızca ağ inference’ının kendisi olan 35,9 ms’lik bir gecikme yaşar; cold-start’taki 8 saniyelik bekleme bir daha ödenmez. Bu yapıdaki tek kısıt sistemden değil, EV-SpSegNet’in 8 saniyelik bir temporal pencereye olan yapısal ihtiyacından kaynaklanır.

Artılar. Kuş, yaprak veya sahne kalabalığı kaynaklı kısa occlusion’lar doğal şekilde absorbe edilir; tracker kilitliken sistem tracker latency’sinde kalır.

6 STC Loss’unun Rolü

STC loss, EV-SpSegNet’e özgü değildir; event başına binary classification yapan herhangi bir model bu spatiotemporal tutarlılık regularizer’ından faydalanabilir. Tablo 2, $\mathcal{L}_{\text{stc}}$ ’nin KPConv, RandLA-Net ve COSeg’e eklendiğinde her üç metrikte tutarlı kazanç ve false alarm rate’inde belirgin düşüş sağladığını göstermektedir [1].

Yöntem	IoU (%) $\uparrow$	ACC (%) $\uparrow$	P_d (%) $\uparrow$	F_a (10^{-4}) $\downarrow$
KPConv	48.19	57.28	68.59	16.32
KPConv + STC	50.32 (+2.13)	59.03 (+1.75)	69.99 (+1.40)	10.60 (−5.72)
RandLA-Net	50.32	59.29	70.56	9.21
RandLA-Net + STC	51.15 (+0.83)	60.63 (+1.34)	72.36 (+2.37)	8.91 (−1.69)
COSeg	51.89	60.93	71.32	6.95
COSeg + STC	53.12 (+1.23)	62.31 (+0.99)	73.96 (+2.64)	5.86 (−1.09)

Tablo 2: STC loss’un farklı backbone’lara eklendiğinde performansa etkisi ([1] Tab. 6). Üç yöntemde de IoU, ACC ve P_d artarken false alarm rate düşmüştür. En çarpıcı kazanç KPConv’da: false alarm rate %35 oranında düşmektedir.

Pipeline için kritik nokta, STC'nin *event-level annotation* gerektirmesidir: her event için $y \in \{0, 1\}$ etiketi olmadan loss çalışmaz. EV-UAV bu gereksinimi bbox'ları zaman ekseninde 3B kutulara genişletip içindeki event'leri target sayan yarı-otomatik bir prosedürle karşılar; aynı strateji kendi verimize de uygulanabilir.

STC'yi farklı backbone'lara taşıma maliyeti, hem annotation yoğunluğuna hem de hedef mimarinin temporal yapıyı ne ölçüde koruduğuna bağlıdır. Point-cloud ağlarında STC sabit, weight'siz bir convolution + sigmoid olduğu için ek değişiklik gerektirmez ve paperdaki ablation üç farklı backbone'da tutarlı kazanç gösterir. FRED gibi YOLO-tabanlı frame detector'lara uygulanması ise iki ayrı uyarılama gerektirir: (i) bbox regression head'in dense bir prediction map (pixel-wise objectness veya segmentation mask) ile desteklenmesi ve etiketlerin buna uygun yoğunlukta sağlanması, (ii) event'lerin frame'e accumulate edilmesiyle azalan temporal çözünürlüğün loss'un komşuluk tanımı üzerindeki etkisinin incelenmesi. Bu ikinci nokta açık bir sorudur: STC'nin sparse event uzayındaki temel avantajı $k \times k \times \tau$ komşuluğunun fiziksel sürekliliği yakalamasıdır ve bu yapının frame-tabanlı temsilde ne kadar korunduğu ampirik olarak doğrulanmalıdır [2].

7 Sonuç

Bu çalışmada EV-SpSegNet [1] mimarisi *ev-drone-detector* reposunda yeniden uygulanmış, end-to-end çalışmayı engelleyen düzeltmeler yapılmış ve mevcut frame-tabanlı FRED pipeline ile arasında bir event-frame köprüsü kurulmuştur. 8 saniyelik receptive field'a sahip bir batch modelin Jetson Orin tabanlı gerçek-zamanlı bir karşı-İHA sistemine entegrasyonu için iki strateji incelenmiştir: sequence başında bir kez çalıştırılan *cold-start* kipi ve track kaybında ring buffer üzerinden devreye giren *re-acquisition* kipi. Her iki kipte de kare-başına yük hafif bir 2B tracker'a bırakılmakta, ağır event-segmentation modeli yalnızca discovery ve recovery aşamalarında devreye girmektedir.

Kaynaklar

- [1] “Event-based Tiny Object Detection: A Benchmark Dataset and Baseline,” *arXiv preprint arXiv:2506.23575*, 2025. <https://doi.org/10.48550/arXiv.2506.23575>
- [2] “Drone Detection with Event Cameras,” *arXiv preprint arXiv:2508.04564*, 2025. <https://arxiv.org/abs/2508.04564>